

ParnaService — это децентрализованная краудфандинговая платформа, где пользователи создают кампании по сбору средств, делают взносы в криптовалюте, а при достижении цели получают долю прибыли пропорционально своим взносам.

1. Глоссарий (Единый язык)

Чтобы исключить двойную трактовку, фиксируем термины:

- **Кампания** — проект по сбору средств, имеющий целевую сумму и дедлайн.
 - **Создатель (Админ кампании)** — пользователь, инициировавший кампанию.
 - **Участник (Вкладчик)** — пользователь, внесший средства в кампанию.
 - **Виртуальные средства** — криптовалюта или токены
 - **Дивиденды (Профит)** — дополнительная сумма, которую Создатель вносит в смарт-контракт после закрытия сбора для распределения среди Участников.
 - **Смарт-контракт (СК)** — код в блокчейне, хранящий средства и выполняющий логику распределения.
-

2. Основная доменная модель (Сущности)

Поскольку архитектура монолитная (Java/Spring Boot), модель будет реляционной (PostgreSQL/MySQL), но с привязкой к блокчейну.

Сущности

1. User (Пользователь)

- `id (UUID)` — уникальный идентификатор
- `walletAddress (String, 42)` — адрес криптокошелька, обязательно для взаимодействия с Web3j, уникальный
- `username (String, 32)` — имя пользователя, уникальное
- `firstName (String, 100)` — имя
- `lastName (String, 100)` — фамилия
- `createdAt (Instant)` — дата создания аккаунта
- `lastLoginAt (Instant)` — дата последнего входа

2. Campaign (Кампания)

- id (UUID) — уникальный идентификатор
- title (String, 120) — название кампании
- shortDescription (String, 300) — краткое описание
- description (String, 5000) — полное описание
- creator (User) — создатель кампании (ManyToOne)
- targetAmount (String, 78) — целевая сумма сбора (в wei для точности)
- raisedAmount (String, 78) — текущая собранная сумма
- status (CampaignStatus) — статус кампании
- deadline (Instant) — дедлайн кампании
- chainId (int) — идентификатор блокчейн-сети
- contractAddress (String, 42) — адрес смарт-контракта в блокчейне
- createdAt (Instant) — дата создания кампании

3. Contribution (Взнос в кампанию)

- id (UUID) — уникальный идентификатор
- campaign (Campaign) — кампания (ManyToOne)
- contributorUser (User) — пользователь-вкладчик (ManyToOne, nullable)
- contributorWallet (String, 42) — адрес кошелька вкладчика
- amount (String, 78) — сумма взноса (в wei)
- status (ContributionStatus) — статус транзакции
- transactionHash (String, 66) — хеш транзакции в блокчейне
- createdAt (Instant) — дата создания взноса
- confirmedAt (Instant) — дата подтверждения транзакции

4. Payout (Выплата профита)

- id (UUID) — уникальный идентификатор
- campaign (Campaign) — кампания (ManyToOne)
- profitAmount (String, 78) — сумма профита для распределения
- status (PayoutStatus) — статус выплаты
- transactionHash (String, 66) — хеш транзакции в блокчейне
- createdAt (Instant) — дата создания выплаты
- confirmedAt (Instant) — дата подтверждения транзакции

5. PayoutDistribution (Распределение выплат участникам)

- id (UUID) — уникальный идентификатор
- payout (Payout) — выплата (ManyToOne)

- `walletAddress` (String, 42) — адрес кошелька получателя
- `contributionAmount` (String, 78) — сумма первоначального взноса
- `sharePercent` (double) — процент доли участника
- `payoutAmount` (String, 78) — сумма выплаты
- `transferStatus` (TransferStatus) — статус перевода
- `transactionHash` (String, 66) — хеш транзакции перевода

6. RefreshToken (Токен обновления сессии)

- `id` (UUID) — уникальный идентификатор
- `user` (User) — пользователь (ManyToOne)
- `tokenHash` (String, 64) — хеш токена, уникальный
- `expiresAt` (Instant) — срок действия токена
- `revoked` (boolean) — отозван ли токен
- `createdAt` (Instant) — дата создания токена

Перечисления (Enums)

CampaignStatus (Статус кампании)

- `PENDING_DEPLOYMENT` — ожидает деплоя смарт-контракта
- `OPEN` — открыта для сбора средств
- `CLOSED_GOAL_REACHED` — закрыта, цель достигнута
- `FAILED_DEADLINE` — завершена без достижения цели

ContributionStatus (Статус взноса)

- `PREPARED` — подготовлен, транзакция ещё не отправлена
- `PENDING` — транзакция отправлена, ожидает подтверждения
- `CONFIRMED` — транзакция подтверждена в блокчейне
- `FAILED` — транзакция не удалась
- `REVERTED` — транзакция откатана

PayoutStatus (Статус выплаты)

- `PREPARED` — подготовлена
- `PENDING` — ожидает подтверждения
- `CONFIRMED` — подтверждена
- `FAILED` — не удалась
- `REVERTED` — откатана

TransferStatus (Статус перевода)

- PENDING — ожидает подтверждения
- CONFIRMED — подтверждён
- FAILED — не удался

TransactionType (Тип транзакции)

- CAMPAIGN_DEPLOYMENT — деплой смарт-контракта кампании
- CONTRIBUTION — взнос в кампанию
- GOAL_REACHED — достижение цели
- CAMPAIGN_FINALIZATION — завершение кампании
- PAYOUT — выплата профита

TransactionStatus (Статус транзакции)

- PREPARED — подготовлена
- SUBMITTED — отправлена
- PENDING — ожидает подтверждения
- CONFIRMED — подтверждена
- FAILED — не удалась
- REVERTED — откатана
- REORGED — затронута реорганизацией блокчейна

Связи между сущностями

User 1 $\longleftrightarrow$ Campaign (создаёт)

User 1 $\longleftrightarrow$ Contribution (делает взносы)

Campaign 1 $\longleftrightarrow$ Contribution (принимает взносы)

Campaign 1 $\longleftrightarrow$ Payout (распределяет профит)

Payout 1 $\longleftrightarrow$ PayoutDistribution (детализирует выплаты)

User 1 $\longleftrightarrow$ RefreshToken (имеет токены сессий)

3. Юзкейсы (Use Cases)

UC-001 Просмотр главной страницы платформы

Цель: Пользователь хочет ознакомиться с платформой и увидеть наиболее интересные/активные кампании.

Предусловия:

- Платформа доступна

Основной сценарий:

1. Пользователь открывает главную страницу платформы
2. Система отображает:
 - название платформы
 - список популярных кампаний
 - список активных кампаний
3. Пользователь просматривает доступные кампании
4. Пользователь может перейти на страницу выбранной кампании

Альтернативные сценарии:

- **A1. Нет активных кампаний:**
 1. Система отображает сообщение об отсутствии активных кампаний
 2. Предлагает воспользоваться поиском или зарегистрироваться для создания новой кампании

Постусловия:

- Пользователь получил обзор доступных кампаний
- Данные платформы не изменены

Критерии приемки:

1. **Отображение элементов страницы:**
 - Название платформы отображается в заголовке
 - Список популярных кампаний отображается (топ-10 по сумме сборов)
 - Список активных кампаний отображается (статус OPEN, отсортирован по дате создания)
2. **Навигация:**
 - Клик по кампании открывает страницу UC-004
 - Кнопка поиска переводит на UC-002
3. **Альтернативный сценарий (нет активных кампаний):**
 - Отображается сообщение "Активные кампании отсутствуют"
 - Отображается кнопка перехода к поиску
 - Отображается кнопка регистрации для создания кампании
4. **Производительность:**

- Страница загружается за ≤ 2 секунды при наличии до 100 кампаний
-

UC-002 Поиск кампаний

Цель: Пользователь хочет найти интересующую его кампанию.

Предусловия:

- Пользователь не авторизован
- Платформа доступна

Основной сценарий:

1. Пользователь вводит поисковый запрос
2. Система принимает поисковый запрос
3. Система выполняет поиск среди доступных кампаний
4. Система отображает список найденных кампаний
5. Пользователь выбирает кампанию для просмотра подробной информации

Возможные параметры поиска:

- название кампании

Альтернативные сценарии:

- **A1. Кампании не найдены:**
 1. Система сообщает, что подходящие кампании отсутствуют
 2. Предлагает изменить поисковый запрос

Постусловия:

- Пользователь получил список подходящих кампаний

Критерии приемки:

1. **Функционал поиска:**
 - Поле ввода принимает текст до 120 символов
 - Поиск выполняется по подстроке в названии кампании (title)
 - Поиск регистронезависимый
 - Результаты отображаются в реальном времени (debounce 300ms) или по кнопке "Найти"
2. **Отображение результатов:**

- Каждая найденная кампания показывает: название, краткое описание, статус
- Клик по кампании открывает UC-004

3. Альтернативный сценарий (ничего не найдено):

- Отображается сообщение "Кампании не найдены"
- Предлагается изменить поисковый запрос
- Отображается ссылка на UC-003 (все кампании)

4. Валидация:

- Пустой поисковый запрос не выполняется
 - Специальные символы экранируются (защита от XSS/SQL injection)
-

UC-003 Просмотр списка кампаний

Цель: Пользователь хочет ознакомиться со всеми доступными кампаниями.

Предусловия:

- Пользователь находится на главной странице или странице поиска
- В системе существуют опубликованные кампании

Основной сценарий:

1. Пользователь открывает раздел "Кампании"
2. Система отображает список кампаний
3. Для каждой кампании отображается:
 - название
 - краткое описание
 - статус
 - целевая сумма
 - текущая сумма сбора
 - дата создания
 - дедлайн
4. Пользователь выбирает кампанию

Альтернативные сценарии:

- **A1. Нет доступных кампаний:**
 - Система отображает пустой список с соответствующим сообщением

Постусловия:

- Пользователь получил информацию о доступных кампаниях

Критерии приемки:

1. Отображение списка:

- Каждая кампания показывает: название, краткое описание (до 300 символов), статус, целевая сумма, текущая сумма, дата создания, дедлайн
- Суммы отображаются в читаемом формате (например, "1.5 ETH" вместо "1500000000000000000 wei")
- Статус отображается цветным бейджем (OPEN - зеленый, CLOSED_GOAL_REACHED - синий, FAILED_DEADLINE - серый)

2. Пагинация:

- По умолчанию отображается 20 кампаний на странице
- Доступна навигация по страницам
- Сортировка по умолчанию: дата создания (новые сверху)

3. Альтернативный сценарий (нет кампаний):

- Отображается сообщение "Кампании отсутствуют"
- Отображается кнопка создания кампании (если пользователь авторизован)

4. Навигация:

- Клик по кампании открывает UC-004

UC-004 Просмотр страницы кампании

Цель: Пользователь хочет получить полную информацию о конкретной кампании.

Предусловия:

- Кампания существует
- Кампания доступна для просмотра

Основной сценарий:

1. Пользователь открывает страницу кампании
2. Система загружает информацию о кампании
3. Система отображает:
 - **Информация о кампании:**
 - название
 - описание
 - автор кампании

- дата создания
- дедлайн
- статус: Открыта / Закрыта / Завершена без достижения цели
- **Финансовая информация:**
 - целевая сумма сбора
 - текущая сумма сбора
 - процент выполнения цели
 - количество участников
- **Информация о выплатах:**
 - список произведённых выплат
 - дата выплаты
 - сумма выплаты
 - общий размер распределённого профита
- **Информация о транзакциях:**
 - история операций в блокчейне
 - ссылки/идентификаторы транзакций (если предусмотрено)

4. Пользователь просматривает данные

Альтернативные сценарии:

- **A1. Кампания не найдена:**
 - Система отображает сообщение "Кампания не найдена"
- **A2. Ошибка получения данных блокчейна:**
 1. Система отображает доступные данные из локального хранилища
 2. Отображает уведомление о временной недоступности полной информации

Постусловия:

- Пользователь получил публичную информацию о кампании

Критерии приемки:

1. **Основная информация:**
 - Отображается: название, полное описание, автор (ссылка на UC-008), дата создания, дедлайн, статус
 - Статус отображается текстом: "Открыта", "Закрыта (цель достигнута)", "Завершена без достижения цели"
2. **Финансовая информация:**
 - Целевая сумма сбора (в ETH/wei)
 - Текущая сумма сбора (в ETH/wei)

- Процент выполнения цели (рассчитывается как $(\text{raisedAmount} / \text{targetAmount}) * 100$, округляется до 2 знаков)
- Количество уникальных участников (количество подтвержденных взносов)

3. Информация о выплатах:

- Если выплаты есть: список с датой, суммой, количеством получателей
- Если выплат нет: сообщение "Выплаты ещё не производились"

4. Информация о транзакциях:

- Отображается contractAddress (если кампания задеплоена)
- Ссылка на блокчейн-эксплорер для просмотра транзакций

5. Альтернативные сценарии:

- **Кампания не найдена:** Отображается 404 страница с сообщением "Кампания не найдена"
- **Ошибка блокчейна:** Отображаются локальные данные + уведомление "Некоторые данные могут быть недоступны"

6. Действия:

- Если статус = OPEN и пользователь авторизован: отображается кнопка "Внести средства" (UC-009)
- Если пользователь = создатель: отображаются дополнительные элементы управления (UC-014)

UC-005 Просмотр истории выплат кампании

Цель: Пользователь хочет проверить прозрачность распределения средств.

Предусловия:

- Кампания существует
- Выплаты ранее выполнялись

Основной сценарий:

1. Пользователь открывает раздел выплат кампании
2. Система получает информацию о выплатах
3. Система отображает:
 - дату выплаты
 - размер распределённой суммы
 - количество получателей
 - информацию о транзакции смарт-контракта

Альтернативные сценарии:

- **A1. Выплаты отсутствуют:**

- Система отображает сообщение: "По данной кампании выплаты ещё не производились"

Постусловия:

- Пользователь получил историю распределения средств

Критерии приемки:

1. **Отображение выплат:**

- Каждая выплата показывает: дата, сумма профита, количество получателей, transactionHash
- Клик по transactionHash открывает блокчейн-эксплорер в новой вкладке

2. **Детализация:**

- Клик по выплате раскрывает список получателей с суммами (PayoutDistribution)

3. **Альтернативный сценарий (нет выплат):**

- Отображается сообщение "По данной кампании выплаты ещё не производились"

4. **Доступ:**

- История выплат доступна всем пользователям (публичная информация)
-

UC-006 Регистрация пользователя

Цель: Пользователь хочет создать аккаунт в платформе, связав его с адресом криптокошелька.

Предусловия:

- Пользователь не авторизован
- У пользователя имеется криптокошелек

Основной сценарий:

1. Пользователь выбирает регистрацию
2. Система предлагает подключить криптокошелек
3. Пользователь подтверждает подключение кошелька
4. Система получает адрес криптокошелька
5. Система проверяет, зарегистрирован ли данный адрес ранее

6. Система отображает форму регистрации
7. Пользователь вводит:
 - username
 - имя
 - фамилию
8. Пользователь подтверждает регистрацию
9. Система создаёт профиль пользователя и связывает его с адресом криптокошелька
10. Система выполняет авторизацию пользователя

Альтернативные сценарии:

- Адрес кошелька уже зарегистрирован → система предлагает выполнить вход
- Пользователь отклонил подключение кошелька → регистрация отменяется
- Username уже занят → система предлагает выбрать другой

Постусловия:

- Создан новый пользовательский профиль
- Адрес криптокошелька привязан к аккаунту
- Пользователь авторизован в системе

Критерии приемки:

1. Подключение кошелька:

- Отображается кнопка "Подключить кошелек"
- Поддерживаются провайдеры: MetaMask, WalletConnect (минимум)
- После подключения отображается адрес кошелька (первые 6 и последние 4 символа)

2. Форма регистрации:

- Обязательные поля: username (3-32 символа, только латиница, цифры, подчеркивание)
- Опциональные поля: firstName (до 100 символов), lastName (до 100 символов)
- Валидация в реальном времени

3. Валидация уникальности:

- Проверка уникальности walletAddress (если уже зарегистрирован → предложение войти)
- Проверка уникальности username (если занят → сообщение "Username уже занят, выберите другой")

4. Успешная регистрация:

- Создается запись в таблице users

- walletAddress привязан к пользователю
- Автоматическая авторизация (выпуск JWT токенов)
- Редирект на главную страницу или личный кабинет

5. Альтернативные сценарии:

- Пользователь отклонил подключение кошелька → регистрация отменяется, сообщение "Подключение кошелька отклонено"
 - Ошибка сети → сообщение "Ошибка подключения, попробуйте снова"
-

UC-007 Авторизация пользователя

Цель: Пользователь хочет войти в платформу с помощью своего криптокошелька.

Предусловия:

- Пользователь зарегистрирован
- Пользователь имеет доступ к привязанному криптокошельку

Основной сценарий:

1. Пользователь выбирает вход через криптокошелек
2. Система запрашивает подключение кошелька
3. Пользователь подтверждает подключение
4. Система получает адрес криптокошелька
5. Система выполняет поиск пользователя по адресу кошелька
6. Система создаёт пользовательскую сессию
7. Пользователь получает доступ к платформе

Альтернативные сценарии:

- Адрес кошелька не найден → система предлагает пройти регистрацию
- Пользователь отклонил подключение кошелька → авторизация отменяется

Постусловия:

- Пользователь успешно авторизован
- Система идентифицирует пользователя по адресу криптокошелька

Критерии приемки:

1. Процесс авторизации:

- Кнопка "Войти через кошелек"

- Запрос подписи сообщения (challenge) от кошелька
- Формат сообщения: "Войти в Parna Platform: [nonce]" (nonce генерируется сервером)

2. Верификация:

- Сервер проверяет подпись сообщения
- Извлекает адрес кошелька из подписи
- Поиск пользователя по walletAddress

3. Успешная авторизация:

- Выпуск access токена (JWT, срок действия 15 минут)
- Выпуск refresh токена (срок действия 7 дней)
- Сохранение хеша refresh токена в таблице refresh_tokens
- Обновление lastLoginAt в таблице users
- Редирект на главную страницу

4. Альтернативные сценарии:

- Адрес кошелька не найден → предложение зарегистрироваться
- Пользователь отклонил подпись → авторизация отменяется
- Истек срок действия nonce → запрос нового nonce

5. Безопасность:

- Refresh токен хранится в БД только в виде хеша (SHA-256)
- Access токен содержит: userId, walletAddress, exp (expiration)

УС-008 Просмотр страницы пользователя

Цель: Пользователь хочет ознакомиться с информацией о создателе кампаний и его активностью на платформе.

Предусловия:

- Пользователь находится на странице пользователя
- Профиль пользователя существует

Основной сценарий:

1. Пользователь открывает страницу пользователя
2. Система загружает информацию о пользователе
3. Система отображает:
 - username
 - имя

- фамилию
- список кампаний, созданных пользователем:
 - название кампании
 - описание
 - статус
 - целевая сумма
 - текущая сумма сбора
 - дата создания

Альтернативные сценарии:

- Пользователь не имеет созданных кампаний → система отображает сообщение "Пользователь ещё не создавал кампаний"
- Профиль пользователя не найден → система отображает ошибку

Постусловия:

- Пользователь получил информацию о профиле и созданных им кампаниях

Критерии приемки:

1. Публичная информация:

- Отображается: username, firstName, lastName
- walletAddress НЕ отображается (приватная информация)

2. Список кампаний:

- Отображаются только кампании со статусом OPEN, CLOSED_GOAL_REACHED, FAILED_DEADLINE
- Для каждой кампании: название, описание (краткое), статус, целевая сумма, текущая сумма, дата создания
- Сортировка: дата создания (новые сверху)

3. Альтернативные сценарии:

- Пользователь не создавал кампаний → сообщение "Пользователь ещё не создавал кампаний"
- Пользователь не найден → 404 страница

4. Навигация:

- Клик по кампании открывает UC-004

UC-009 Внесение средств в кампанию

Цель: Пользователь хочет поддержать выбранную кампанию, отправив виртуальные средства.

Предусловия:

- Пользователь авторизован
- Кампания существует
- Кампания находится в статусе "Открыта"
- Цель сбора ещё не достигнута

Основной сценарий:

1. Пользователь открывает страницу кампании
2. Система отображает кнопку "Внести средства"
3. Пользователь указывает сумму взноса
4. Пользователь подтверждает операцию
5. Система выполняет перевод средств в рамках кампании
6. Система получает подтверждение успешной транзакции
7. Система обновляет информацию о кампании:
 - увеличивает текущую сумму сбора
 - обновляет количество участников
 - сохраняет информацию о взносе пользователя
8. Пользователь получает уведомление об успешном внесении средств

Альтернативные сценарии:

- Пользователь указал некорректную сумму → система отображает ошибку
- Кампания уже закрыта → внесение средств недоступно
- Целевая сумма достигнута во время операции → система отклоняет взнос

Постусловия:

- Средства пользователя учтены в кампании
- Информация о транзакции сохранена

Критерии приемки:

1. **Предусловия:**
 - Пользователь авторизован
 - Кампания имеет статус OPEN
 - `raisedAmount < targetAmount` (цель не достигнута)
 - `deadline > текущая дата`

2. Процесс внесения:

- Кнопка "Внести средства" открывает модальное окно
- Поле ввода суммы (валидация: > 0 , не превышает остаток до цели)
- Отображается текущий баланс кошелька пользователя
- Кнопка "Подтвердить" запрашивает подпись транзакции от кошелька

3. Транзакция:

- Создается запись Contribution со статусом PREPARED
- Отправляется транзакция в блокчейн
- Сохраняется transactionHash
- Статус меняется на PENDING

4. Подтверждение:

- После подтверждения транзакции в блокчейне:
 - Contribution.status = CONFIRMED
 - Campaign.raisedAmount увеличивается на сумму взноса
 - Создается уведомление для пользователя

5. Альтернативные сценарии:

- Некорректная сумма → ошибка валидации
- Кампания закрылась во время операции → сообщение "Кампания закрыта, взнос отклонен"
- Цель достигнута во время операции → сообщение "Цель достигнута, взнос отклонен"
- Пользователь отклонил транзакцию → взнос отменяется
- Ошибка транзакции → Contribution.status = FAILED, сообщение "Ошибка транзакции"

6. Идемпотентность:

- Повторная отправка той же транзакции не создает дублирующую запись

UC-010 Просмотр личного кабинета

Цель: Пользователь хочет просмотреть информацию о своей активности на платформе.

Предусловия:

- Пользователь авторизован

Основной сценарий:

1. Пользователь открывает личный кабинет

2. Система загружает информацию о пользователе
3. Система отображает:
 - username
 - имя
 - фамилию
 - адрес криптокошелька
4. Система отображает список кампаний пользователя:
 - кампании, в которые пользователь внес средства
 - созданные пользователем кампании
5. Пользователь может перейти на страницу выбранной кампании

Постусловия:

- Пользователь получил информацию о своей активности

Критерии приемки:

1. **Информация о пользователе:**
 - Отображается: username, firstName, lastName, walletAddress (полный адрес)
 - Кнопка копирования walletAddress
2. **Список кампаний:**
 - **Внесенные средства:** кампании, в которые пользователь делал взносы (статус CONFIRMED)
 - Для каждой: название, статус, сумма взноса, дата
 - **Созданные кампании:** кампании, созданные пользователем
 - Для каждой: название, статус, целевая сумма, текущая сумма, дата создания
3. **Навигация:**
 - Клик по кампании открывает UC-004 или UC-014 (если пользователь = создатель)
4. **Альтернативные сценарии:**
 - Пользователь не участвовал в кампаниях → пустой список с сообщением

UC-011 Просмотр истории внесённых средств

Цель: Пользователь хочет посмотреть информацию о своих взносах в кампании.

Предусловия:

- Пользователь авторизован

Основной сценарий:

1. Пользователь открывает раздел своих вкладов в личном кабинете
2. Система получает список операций пользователя
3. Система отображает:
 - название кампании
 - дату внесения средств
 - сумму взноса
 - статус кампании
 - идентификатор транзакции
4. Пользователь выбирает кампанию для просмотра деталей

Альтернативные сценарии:

- Пользователь не имеет внесённых средств → система отображает пустой список

Постусловия:

- Пользователь получил историю своих вложений

Критерии приемки:

1. **Отображение истории:**
 - Список всех взносов пользователя (статус CONFIRMED)
 - Для каждого взноса: название кампании, дата, сумма, статус кампании, transactionHash
 - Сортировка: дата (новые сверху)
2. **Детализация:**
 - Клик по transactionHash открывает блокчейн-эксплорер
 - Клик по кампании открывает UC-004
3. **Альтернативные сценарии:**
 - Нет взносов → пустой список с сообщением "Вы ещё не вносили средства"
4. **Фильтрация:**
 - Опционально: фильтр по статусу кампании (OPEN, CLOSED, FAILED)

UC-012 Просмотр своих созданных кампаний

Цель: Пользователь хочет посмотреть список кампаний, созданных им ранее.

Предусловия:

- Пользователь авторизован
- Пользователь имеет созданные кампании

Основной сценарий:

1. Пользователь открывает раздел своих кампаний
2. Система получает список кампаний пользователя
3. Система отображает:
 - название кампании
 - описание
 - статус
 - целевую сумму
 - текущую сумму сбора
 - дату создания
 - дедлайн
4. Пользователь может перейти на страницу выбранной кампании

Альтернативные сценарии:

- Пользователь не создавал кампаний → система отображает сообщение об отсутствии кампаний

Постусловия:

- Пользователь получил список своих кампаний

Критерии приемки:

1. **Отображение списка:**
 - Все кампании, созданные пользователем
 - Для каждой: название, описание (краткое), статус, целевая сумма, текущая сумма, дата создания, дедлайн
 - Сортировка: дата создания (новые сверху)
 2. **Навигация:**
 - Клик по кампании открывает UC-014 (управление кампанией)
 3. **Альтернативные сценарии:**
 - Нет созданных кампаний → сообщение "Вы ещё не создавали кампаний" + кнопка "Создать кампанию"
-

UC-013 Создание кампании

Цель: Пользователь хочет создать новую кампанию для сбора виртуальных средств.

Предусловия:

- Пользователь авторизован
- Пользователь имеет подключенный криптокошелёк

Основной сценарий:

1. Пользователь открывает форму создания кампании
2. Система отображает форму создания кампании
3. Пользователь вводит данные:
 - название кампании
 - описание
 - целевую сумму сбора
 - дедлайн кампании
4. Пользователь подтверждает создание кампании
5. Система проверяет введённые данные
6. Система создаёт кампанию
7. Система связывает кампанию с пользователем-создателем
8. Система отображает созданную кампанию

Альтернативные сценарии:

- Не заполнены обязательные поля → система отображает ошибки валидации
- Целевая сумма или дедлайн некорректны → система отклоняет создание
- Пользователь отменяет создание → кампания не создаётся

Постусловия:

- Создана новая кампания
- Пользователь назначен её администратором
- Кампания доступна для внесения средств

Критерии приемки:

1. **Форма создания:**
 - Обязательные поля:
 - title (1-120 символов)
 - targetAmount (> 0, в wei)

- deadline (дата > текущей даты)
- Опциональные поля:
 - shortDescription (до 300 символов)
 - description (до 5000 символов)

2. Валидация:

- Все обязательные поля заполнены
- targetAmount > 0
- deadline > текущая дата + минимум 1 день
- title не содержит запрещенных символов

3. Процесс создания:

- Создается запись Campaign со статусом PENDING_DEPLOYMENT
- creator_id = ID текущего пользователя
- createdAt = текущая дата
- raisedAmount = "0"
- chainId = ID текущей сети (из конфигурации)

4. Деплой смарт-контракта:

- Асинхронно запускается процесс деплоя
- После подтверждения: contractAddress сохраняется, статус меняется на OPEN
- Пользователь получает уведомление о готовности кампании

5. Альтернативные сценарии:

- Ошибки валидации → отображение ошибок под полями
- Ошибка деплоя → статус остается PENDING_DEPLOYMENT, уведомление об ошибке
- Пользователь отменил создание → кампания не создается

6. Постусловия:

- Кампания создана
- Пользователь перенаправляется на UC-014

UC-014 Просмотр управления собственной кампанией

Цель: Администратор хочет получить доступ к функциям управления своей кампанией.

Предусловия:

- Пользователь авторизован
- Пользователь является создателем кампании

Основной сценарий:

1. Пользователь открывает страницу своей кампании
2. Система проверяет права пользователя
3. Система отображает расширенную информацию:
 - текущий статус кампании
 - целевую сумму
 - собранную сумму
 - количество участников
 - историю внесённых средств
 - доступные действия управления

Постусловия:

- Пользователь получил доступ к управлению своей кампанией

Критерии приемки:

1. **Доступ:**
 - Только если пользователь = creator кампании
 - Иначе редирект на UC-004 (публичный просмотр)
 2. **Дополнительная информация:**
 - Текущий статус кампании
 - Целевая и собранная суммы
 - Количество участников
 - История всех взносов (с адресами кошельков)
 - Статус деплоя смарт-контракта
 3. **Действия управления:**
 - Если статус = CLOSED_GOAL_REACHED и выплата не производилась:
 - Кнопка "Распределить профит" (UC-015)
 - Если статус = PENDING_DEPLOYMENT:
 - Индикатор прогресса деплоя
 - Если статус = OPEN:
 - Кнопка "Завершить кампанию досрочно" (опционально)
 4. **Статистика:**
 - График сбора средств (опционально)
 - Количество дней до дедлайна
-

УС-015 Внесение профита и распределение выплат участникам

Цель: Администратор хочет внести полученную прибыль по завершённой кампании, чтобы система автоматически распределила её между участниками.

Предусловия:

- Пользователь авторизован
- Пользователь является создателем кампании
- Кампания закрыта
- Кампания достигла целевой суммы
- Выплата ранее не производилась

Основной сценарий:

1. Администратор открывает страницу своей закрытой кампании
2. Система отображает возможность внесения профита
3. Администратор вводит сумму профита
4. Администратор подтверждает операцию
5. Система проверяет корректность суммы
6. Система рассчитывает долю каждого участника пропорционально внесённым средствам
7. Система выполняет распределение средств между участниками
8. Система сохраняет информацию о выплате
9. Статус выплаты отображается на странице кампании

Альтернативные сценарии:

- Профит не указан или сумма некорректна → система отображает ошибку
- Выплата уже была выполнена → повторное распределение запрещено
- Кампания не достигла целевой суммы → возможность выплаты недоступна

Постусловия:

- Профит распределён между участниками
- История выплат обновлена
- Информация о распределении доступна пользователям

Критерии приемки:

1. **Предусловия:**
 - Пользователь = creator кампании

- Статус кампании = CLOSED_GOAL_REACHED
- raisedAmount >= targetAmount
- Выплата ранее не производилась (нет записи Payout с статусом CONFIRMED)

2. Форма внесения профита:

- Поле ввода суммы профита (> 0)
- Отображается информация: количество участников, общая сумма взносов
- Предварительный расчет долей участников

3. Расчет долей:

- Для каждого участника:
 - $sharePercent = (contributionAmount / raisedAmount) * 100$
 - $payoutAmount = profitAmount * sharePercent / 100$
- Сумма всех payoutAmount = profitAmount (с точностью до wei)

4. Процесс распределения:

- Создается запись Payout со статусом PREPARED
- Создаются записи PayoutDistribution для каждого участника
- Отправляются транзакции распределения в блокчейн
- Статусы обновляются по мере подтверждения

5. Подтверждение:

- После подтверждения всех транзакций:
 - Payout.status = CONFIRMED
 - Все PayoutDistribution.transferStatus = CONFIRMED
 - Сохраняются transactionHash

6. Альтернативные сценарии:

- Некорректная сумма профита → ошибка валидации
- Выплата уже производилась → сообщение "Выплата уже выполнена"
- Кампания не достигла цели → кнопка недоступна
- Ошибка транзакции → частичное распределение (некоторые выплаты успешны, другие нет)

7. Постусловия:

- Профит распределен
- История выплат обновлена
- Участники получают уведомления

UC-016 Завершение кампании без достижения цели

Цель: Система должна корректно обработать ситуацию, когда дедлайн кампании истёк, но целевая сумма не была достигнута.

Предусловия:

- Кампания имеет статус "Открыта"
- Текущая дата превышает дедлайн
- Целевая сумма не достигнута

Основной сценарий:

1. Система проверяет состояние кампании
2. Система обнаруживает истечение дедлайна
3. Система изменяет статус кампании на "Завершена без достижения цели"
4. Система блокирует дальнейшее внесение средств
5. Система сохраняет текущие данные кампании

Альтернативные сценарии:

- Целевая сумма достигнута до дедлайна → кампания закрывается успешно
- Дедлайн ещё не наступил → никаких изменений не выполняется

Постусловия:

- Кампания завершена
- Новые взносы запрещены
- Выплата профита недоступна

Критерии приемки:

1. **Автоматическая проверка:**
 - Периодическая задача (cron/scheduler) каждые 5 минут
 - Проверяет кампании со статусом OPEN
 - Условие: `deadline < текущая дата AND raisedAmount < targetAmount`
2. **Изменение статуса:**
 - Статус меняется на `FAILED_DEADLINE`
 - Блокируются новые взносы (UC-009 возвращает ошибку)
3. **Уведомления:**
 - Создатель кампании получает уведомление
 - Все участники получают уведомление
4. **Альтернативные сценарии:**
 - Цель достигнута до дедлайна → статус `CLOSED_GOAL_REACHED` (UC-022)

- Дедлайн еще не наступил → никаких изменений

5. Постусловия:

- Кампания завершена
 - Новые взносы запрещены
 - Распределение профита недоступно
-

4. Функциональные требования

1. Модуль аутентификации и управления пользователями

FR-001 Регистрация пользователя через криптокошелек

Описание: Система должна позволять новому пользователю зарегистрироваться, подключив криптокошелек. **Детали:**

- Подключение кошелька через Web3-провайдер (MetaMask, WalletConnect и т.д.)
- Валидация адреса кошелька (формат Ethereum, 42 символа)
- Проверка уникальности адреса кошелька в системе
- Проверка уникальности username (до 32 символов)
- Обязательные поля: username, walletAddress
- Опциональные поля: firstName, lastName **Связь:** UC-006

FR-002 Авторизация через криптокошелек

Описание: Система должна аутентифицировать пользователя по адресу криптокошелька. **Детали:**

- Подпись сообщения (challenge) приватным ключом кошелька
- Поиск пользователя по адресу кошелька
- Выпуск JWT (access + refresh токенов)
- Сохранение хеша refresh токена в БД
- Обновление поля lastLoginAt **Связь:** UC-007

FR-003 Обновление access токена

Описание: Система должна предоставлять новый access токен по валидному refresh токену. **Детали:**

- Проверка хеша refresh токена в БД
- Проверка срока действия (expiresAt)

- Проверка флага revoked
- Ротация refresh токена при использовании **Связь:** UC-007

FR-004 Выход из системы (logout)

Описание: Система должна отзывать все refresh токены пользователя. **Детали:**

- Установка флага revoked = true для всех токенов пользователя
- Очистка сессии на клиенте **Связь:** UC-007

FR-005 Просмотр профиля пользователя

Описание: Система должна отображать публичную информацию о пользователе.

Детали:

- Отображение: username, firstName, lastName
- Список кампаний, созданных пользователем (название, описание, статус, суммы, дата)
- Обработка случая отсутствия кампаний **Связь:** UC-008

FR-006 Просмотр личного кабинета

Описание: Система должна отображать приватную информацию авторизованного пользователя. **Детали:**

- Отображение: username, firstName, lastName, walletAddress
- Список кампаний, в которые пользователь вносил средства
- Список созданных пользователем кампаний
- Возможность перехода на страницу кампании **Связь:** UC-010

2. Модуль просмотра кампаний

FR-007 Просмотр главной страницы

Описание: Система должна отображать обзор активных и популярных кампаний.

Детали:

- Отображение названия платформы
- Список популярных кампаний (по сумме сборов или количеству участников)
- Список активных кампаний (со статусом OPEN)
- Обработка случая отсутствия активных кампаний **Связь:** UC-001

FR-008 Поиск кампаний

Описание: Система должна предоставлять поиск компаний по названию. **Детали:**

- Поиск по подстроке в поле title
- Регистронезависимый поиск
- Отображение списка найденных компаний
- Обработка случая отсутствия результатов **Связь:** UC-002

FR-009 Просмотр списка всех компаний

Описание: Система должна отображать все опубликованные компании с фильтрацией.

Детали:

- Отображение для каждой компании: название, краткое описание, статус, целевая сумма, текущая сумма, дата создания, дедлайн
- Пагинация результатов
- Обработка случая отсутствия компаний **Связь:** UC-003

FR-010 Просмотр детальной страницы компании

Описание: Система должна отображать полную информацию о компании. **Детали:**

- Основная информация: название, описание, автор, даты, статус
 - Финансовая информация: целевая сумма, текущая сумма, процент выполнения, количество участников
 - История выплат (если есть)
 - История транзакций в блокчейне
 - Обработка случая недоступности блокчейн-данных (fallback на локальные данные)
- Связь:** UC-004

FR-011 Просмотр истории выплат компании

Описание: Система должна отображать информацию о распределении профита.

Детали:

- Список выплат: дата, сумма, количество получателей
- Информация о транзакции смарт-контракта
- Обработка случая отсутствия выплат **Связь:** UC-005

3. Модуль создания и управления компаниями

FR-012 Создание новой компании

Описание: Система должна позволять авторизованному пользователю создать кампанию. **Детали:**

- Обязательные поля: title (до 120 символов), targetAmount, deadline
- Опциональные поля: shortDescription (до 300), description (до 5000)
- Валидация: targetAmount > 0, deadline > текущая дата
- Автоматическая привязка к creator_id
- Начальный статус: PENDING_DEPLOYMENT
- Инициация деплоя смарт-контракта в блокчейне **Связь:** UC-013

FR-013 Деплой смарт-контракта кампании

Описание: Система должна автоматически деплоить смарт-контракт после создания кампании. **Детали:**

- Асинхронная отправка транзакции деплоя
- Сохранение contractAddress после подтверждения
- Обновление статуса на OPEN после успешного деплоя
- Обработка ошибок деплоя (статус FAILED) **Связь:** UC-013

FR-014 Просмотр управления собственной кампанией

Описание: Система должна предоставлять расширенные функции управления для создателя кампании. **Детали:**

- Проверка прав доступа (только creator)
- Отображение: статус, суммы, количество участников, история взносов
- Доступ к действиям управления (внесение профита) **Связь:** UC-014

FR-015 Просмотр списка своих кампаний

Описание: Система должна отображать все кампании, созданные пользователем.

Детали:

- Отображение: название, описание, статус, суммы, даты
- Обработка случая отсутствия кампаний **Связь:** UC-012

4. Модуль внесения средств

FR-016 Внесение средств в кампанию

Описание: Система должна позволять авторизованному пользователю внести средства в кампанию. **Детали:**

- Предусловия: статус кампании = OPEN, цель не достигнута
- Валидация суммы (> 0 , не превышает остаток до цели)
- Создание записи Contribution со статусом PREPARED
- Отправка транзакции в блокчейн
- Обновление статуса на PENDING
- После подтверждения: обновление raised_amount в Campaign, смена статуса на CONFIRMED
- Сохранение transactionHash **Связь:** UC-009

FR-017 Обработка подтверждения транзакции взноса

Описание: Система должна отслеживать подтверждение транзакций взносов. **Детали:**

- Мониторинг статуса транзакции в блокчейне
- Обновление Contribution.status при подтверждении/откате
- Обновление Campaign.raised_amount при подтверждении
- Обработка reorg (REORGED статус) **Связь:** UC-009

FR-018 Просмотр истории своих взносов

Описание: Система должна отображать все взносы пользователя. **Детали:**

- Отображение: название кампании, дата, сумма, статус, transactionHash
- Сортировка по дате (новые сверху)
- Возможность перехода на страницу кампании **Связь:** UC-011

5. Модуль распределения профита

FR-019 Внесение профита и распределение выплат

Описание: Система должна позволять создателю кампании распределить полученную прибыль. **Детали:**

- Предусловия: статус = CLOSED_GOAL_REACHED, выплата ранее не производилась
- Валидация суммы профита (> 0)
- Расчёт доли каждого участника: $sharePercent = (contributionAmount / raisedAmount) * 100$
- Расчёт суммы выплаты: $payoutAmount = profitAmount * sharePercent / 100$
- Создание записи Payout со статусом PREPARED
- Создание записей PayoutDistribution для каждого участника
- Отправка транзакций распределения в блокчейн **Связь:** UC-015

FR-020 Обработка подтверждения транзакций выплат

Описание: Система должна отслеживать подтверждение транзакций распределения профита. **Детали:**

- Обновление Payout.status при подтверждении
- Обновление PayoutDistribution.transferStatus для каждой записи
- Сохранение transactionHash
- Обработка частичных подтверждений (некоторые выплаты успешны, другие нет)

Связь: UC-015

6. Системные (автоматические) функции

FR-021 Автоматическое завершение кампании по дедлайну

Описание: Система должна автоматически закрывать кампании, не достигшие цели к дедлайну. **Детали:**

- Периодическая проверка (cron/scheduler) кампаний со статусом OPEN
- Условие: `deadline < текущая дата AND raisedAmount < targetAmount`
- Изменение статуса на FAILED_DEADLINE
- Блокировка новых взносов
- Сохранение текущих данных **Связь:** UC-016

FR-022 Автоматическое закрытие кампании при достижении цели

Описание: Система должна автоматически закрывать кампанию при достижении целевой суммы. **Детали:**

- Проверка при каждом подтверждённом взносе
- Условие: `raisedAmount >= targetAmount`
- Изменение статуса на CLOSED_GOAL_REACHED
- Блокировка новых взносов **Связь:** UC-009, UC-016

7. Интеграция с блокчейном

FR-023 Мониторинг транзакций

Описание: Система должна отслеживать статус всех отправленных транзакций. **Детали:**

- Отслеживание транзакций типов: CAMPAIGN_DEPLOYMENT, CONTRIBUTION, PAYOUT

- Обновление статусов: PREPARED → SUBMITTED → PENDING → CONFIRMED/FAILED/REVERTED
- Обработка reorg (REORGED)
- Retry-механизм для зависших транзакций **Связь:** Все UC с блокчейн-операциями

FR-024 Синхронизация данных с блокчейном

Описание: Система должна обеспечивать консистентность между локальной БД и блокчейном. **Детали:**

- Периодическая сверка статусов транзакций
 - Fallback-режим при недоступности блокчейн-ноды
 - Уведомление пользователей о временной недоступности данных **Связь:** UC-004 (A2)
-

4. Технические спецификации и ограничения (Для разработчиков)

1. **Архитектура:** Монолит (Spring Boot). Фронтенд (HTML/CSS/JS) может быть либо на Thymeleaf, либо SPA, общающаяся с бэком по REST API.
2. **Интеграция с блокчейном (Web3j):**
 - Бэкенд не хранит приватные ключи пользователей. Все транзакции (кроме деплоя, если деплоит бэкенд) подписываются на фронтенде через Web3-провайдер (MetaMask).
 - **Синхронизация состояния:** БД выступает как кэш для UI. Бэкенд должен использовать `Web3j.subscribeToPendingTransactions` или парсить логи блоков, чтобы обновлять статусы кампаний в БД, как только они меняются в блокчейне.